1. Can you explain your application's deployment architecture: servers, load balancers, database, caching, etc.?

A: Our architecture is based on **microservices** running in **Docker containers** managed by **Kubernetes**. All external traffic first hits a **Cloud Load Balancer** (like AWS ALB), which routes it to the **API Gateway**. The gateway then sends requests to the correct service pods. Data is stored in a highly available database (e.g., PostgreSQL RDS), and we use **Redis** as a centralized, distributed cache layer to handle high read traffic.

2. How do you create a Jenkins pipeline from scratch using a Jenkinsfile?

A: You create a Jenkins pipeline by writing a **Jenkinsfile** (which is a script) and checking it into your Git repository. The file uses a **Declarative Pipeline** syntax, defining stages like Build, Test, Docker Build, and Deploy. When a code change is pushed, Jenkins automatically detects the file and runs the stages sequentially from top to bottom.

3. Describe a real example where you automated deployments in a microservices system.

A: We automated deployments using **Jenkins** and **Kubernetes**. When a developer merged code to the main branch, Jenkins automatically started the pipeline. It ran tests, built a new **Docker image**, tagged it with a unique version, and pushed it to our private registry. Finally, it updated the Kubernetes deployment file, triggering a safe, automatic rollout of the new version across the clusters.

4. How do you ensure 20 microservices deploy in the correct order without breaking dependencies?

A: We ensure order by using an **orchestration tool** or script that explicitly defines the dependency order. For example, the User-DB service must deploy before the Auth-Service can deploy. If using Kubernetes, we use **init containers** or health check logic to ensure the dependency is fully healthy before the dependent service starts.

5. How would you investigate a memory leak in a running microservice after deployment?

A: I would investigate by monitoring the microservice's **JVM heap usage** using tools like **JMX** or **Prometheus/Grafana**. If the memory usage continuously increases without dropping after

Garbage Collection, I would take a **Heap Dump** (a memory snapshot). Then, I would analyze that dump using **Eclipse MAT** to find the object that is consuming memory and fix the code holding onto it.

6. During rolling deployments, users see inconsistent behavior. What could be causing it?

A: Inconsistent behavior is typically caused by **API version drift** or **incompatible data changes**. Since both the old version and the new version of the code are running simultaneously, the new version must be fully **backward compatible**. If the new version changes the API contract or data structure in a way the old version cannot handle, users hitting the old service see errors.